SIMPLIFIED *YOLO-Style* OBJECT DETECTOR FROM SCRATCH WITH FOCUS ON CLASSIFICATION

TECHNICAL REPORT

Predrag Zivanovic **Eduard Munteanu**

December 5, 2025

ABSTRACT

Object detection models often struggle with small, minority-class objects embedded in cluttered scenes. We study this setting in a simplified YOLO-style grid classifier trained from scratch to distinguish between football players and the football itself, using a lightweight convolutional backbone and a purely classification-based head. The dataset is heavily imbalanced at the grid-cell level, with ball cells being more than two orders of magnitude rarer than player cells, causing a baseline model to achieve high overall accuracy but poor ball recognition. We systematically evaluate several imbalance-mitigation strategies, including class-weighted loss, random oversampling, a hybrid sampling scheme that jointly oversamples ball-containing images and lightly undersamples player-only images, cyclic learning rates selected via a learning-rate range test, and a hard negative mining stage. While class weighting and naive oversampling yield limited gains, the combination of hybrid sampling, cyclic learning, and hard negative mining raises ball accuracy from below 56% to above 94%, while maintaining player accuracy above 99% and overall accuracy around 97%. These results show that carefully designed sampling and optimization strategies can substantially improve minority-class performance even in very compact YOLO-style architectures focused solely on classification.

1 Introduction

Object detection in unconstrained environments is a challenging task within computer vision. The problem becomes even more demanding when the goal is to detect *small objects* or *rare subclasses* embedded in cluttered scenes. In such settings, the model must resolve subtle spatial cues while coping with large intra-class variation and severe class imbalance. This project focuses on detecting football players and, in particular, identifying the *football*, which represents a small and infrequent object within the dataset. Small-object detection is well known to be significantly more difficult compared to detecting large, well-defined regions due to limited resolution, low signal-to-noise ratio, and the inherent loss of fine detail introduced by convolutional downsampling layers [1]. Modern object detectors such as the "You Only Look Once" (YOLO) family of models are widely recognized for their efficiency, real-time inference speed, and end-to-end detection pipeline [2]. However, even state-of-the-art YOLO variants struggle with small-object recognition unless additional strategies such as data augmentation, oversampling, or class-balanced loss functions are introduced [5]. Many high-performing solutions rely on complex multi-scale feature fusion or computationally heavy architectures, which increase model size and training time. In this project, we investigate whether a simple, lightweight YOLO-style model, built from first principles, can handle the challenge of detecting both players and a minority-class small object (the football). We begin with a minimal YOLOv1-inspired architecture and iteratively improve the model using motivated experiments, including data augmentation, oversampling, and weighted loss functions, while intentionally maintaining architectural simplicity.

Our objective is to evaluate how far a compact detector can be pushed through principled improvements alone—without resorting to heavyweight modern architectures. In doing so, we aim to understand the limitations of lightweight YOLO-style models in small-object detection and identify which interventions yield the greatest performance gains.

2 Methods

In this section, we will present the baseline model that we started with and how this was gradually improved upon from several experiments.

2.1 Dataset

The dataset used was **football-detection-3** which contains images of players and balls on the football pitch with total of 1232 images of the same size collected from various football matches. Each of the classes have 968 images where the both classes are present, and the dataset comes only in one resolution. The dataset is supplemented by two annotations containing bounding boxes for the ball and the player on every image. When training we have used an 80-10-10 training-validation-test split and batch size of 10. Some of the data that we found in the dataset was also mislabeled so we've manually reviewed dataset and removed the mislabeled data.

2.2 Network architecture

The baseline model used self-implemented Simple Yolo backbone that is trained on the dataset, where we have players and footballs. Our baseline model is a lightweight convolutional classifier inspired by the grid-based formulation of YOLO. Instead of producing full bounding-box predictions, the model is adapted for pure classification by predicting, for each spatial cell in an $S \times S$ grid, the class identity of the object present. The architecture consists of a compact convolutional backbone followed by a small prediction head. The backbone is implemented as a shallow sequence of convolution–batch normalization–ReLU layers, which extract mid-level spatial features while maintaining computational efficiency. These feature maps are then passed to a classification head composed of a final convolutional layer that outputs class logits for each grid cell. The head produces a C-dimensional vector per cell, where C denotes the number of classes, and the model is trained using mean-squared error over all positive object cells. This minimalistic grid-based classifier serves as our starting point. It captures only the essential elements needed for classification while intentionally avoiding heavyweight architectures or multi-scale pipelines. Subsequent experiments extend this baseline with class reweighting, oversampling strategies, and task-specific data augmentation to improve performance on the minority class.

Figure 1: Overview of our lightweight grid-based classifier. A compact convolutional backbone extracts spatial features from the input image, which are then mapped by a simple 1×1 convolutional classification head to produce class logits for each cell in an $S \times S$ grid.

2.3 Model evaluation

To evaluate the model we have focused on two parameters. The first being the accuracy of the validation set, the second being the loss function of the model. Accuracy and loss function are interesting metrics as they allow us to reason about the model's performance. In addition to these metrics, we also examine the confusion matrix, which provides a more detailed view of how the model performs across different classes. Unlike accuracy, which summarises performance with a single number, the confusion matrix highlights specific patterns of correct and incorrect predictions, making it possible to identify classes the model struggles with and potential sources of systematic error.

2.4 Initial Training

Before applying any corrective strategy, we first trained our simplified YOLO v1 classifier using only the raw dataset. The model predicts one class per grid–cell (Ball or Player) and uses a lightweight backbone consisting of two convolutional blocks:

This small architecture was chosen intentionally to keep the analysis focused on the classification behaviour rather than network capacity. The model was trained for 50 epochs using Adam with a learning rate of 10^{-3} and 28x28 grids.

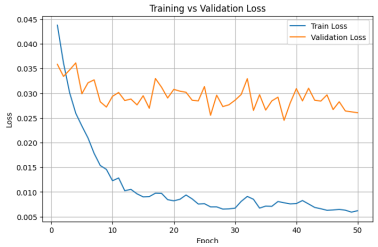

Figure 2: Training vs Validation Loss graph

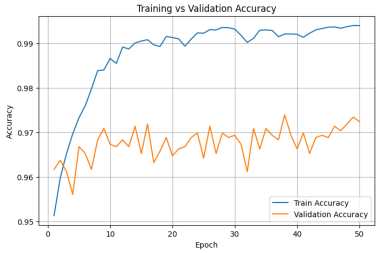

Figure 3: Training vs Validation Accuracy graph

We observe only mild overfitting: the validation loss decreases briefly and then stabilizes at a slightly higher level than the training loss, while the training loss continues to fall throughout the epochs. However, this overfitting is not the main concern. Both losses remain low overall, and the model achieves a high accuracy in the range of 92–100%, suggesting that its general performance is strong. What requires further investigation is the possibility of class imbalance: the dataset contains far more Player examples than Ball examples, and a high overall accuracy may hide poor performance on the minority class. To verify this, we examine the class-specific behaviour using a confusion matrix, as shown below.

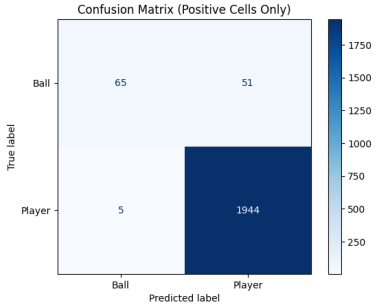

Figure 4: Confusion matrix

We can clearly conclude that the dataset exhibits a severe class imbalance: the number of Player instances is more than 100 times higher than the number of Ball instances. As a consequence, the model learns to predict the majority class in most cases, leading to a deceptively high overall accuracy. The confusion matrix confirms this behaviour: although 86 Ball cells appear in the test set, the model predicts Player for 56 of them. This shows that the performance problem is not global accuracy, but specifically the inability to recognise the minority class.

2.5 Experiments

The experiments were conducted sequentially where the result of one experiment would affect the intention of the next.

2.5.1 Class Imbalance

Class Weighted Loss

To address this issue, we modified the classification loss so that misclassifying a Ball cell is penalised more heavily than misclassifying a Player cell. The weighting factor is proportional to the inverse class frequency in the dataset, making the model place greater emphasis on the minority class. The results of this experiment are shown below, regarding loss and accuracy:

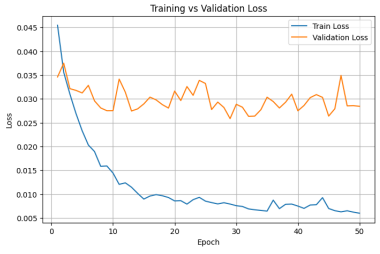

Figure 5: Training vs Validation Loss graph

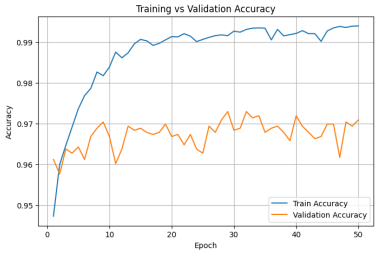

Figure 6: Training vs Validation Accuracy graph

So now we take a look at the confusion matrix again in order to see if any potential improvement happened, then we can decide the next step:

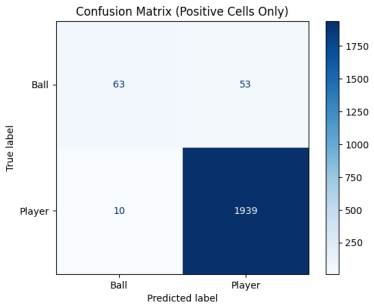

Figure 7: Confusion matrix

Although we applied class-weighted loss to reduce the imbalance between balls and players, the confusion matrix shows almost no improvement. A closer inspection of the dataset explains why. Most images contain players, and many contain players together with a ball. As a result, even when we increase the weight of the ball class, the batches processed during training overwhelmingly contain player-dominant cells.

Because we use a batch size of 10, the probability that a batch contains a meaningful number of positive ball cells is very low. And even when balls do appear, they are typically accompanied by several player cells in the same images. This makes the weighted loss fire only rarely and weakly: the model is almost never exposed to batches where ball examples dominate or even balance the signal from the player class. Consequently, the weighting does not significantly alter the gradient direction, and the model remains biased toward predicting "player".

3 Hybrid Sampling for Class Imbalance

Deep learning models often struggle when the training distribution is dominated by one class, leading to biased decision boundaries and poor generalization on minority classes. A common approach to alleviate this issue is *random oversampling*, where minority-class samples are duplicated during training in order to increase their representation. More sophisticated techniques include SMOTE, ADASYN, or focal loss, all of which attempt to either synthetically generate minority samples or reweight the loss landscape to emphasize difficult examples.

In our dataset, the imbalance manifests not at the image level but at the level of *positive grid cells*. Images containing a football also contain between two and eleven players, whereas many images contain only players. Pure oversampling of ball-containing images therefore implicitly increases the number of player-positive cells as well; oversampling does not remove the majority class signal but rather reshapes the ratio of "ball" to "player" grid cells. Despite this, oversampling alone does not fully correct the imbalance, because player-only images continue to dominate the training distribution as we can see in Figure 8.

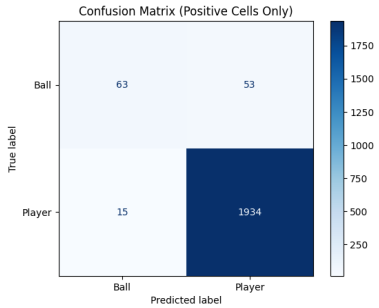

Figure 8: Confusion matrix for Overampling

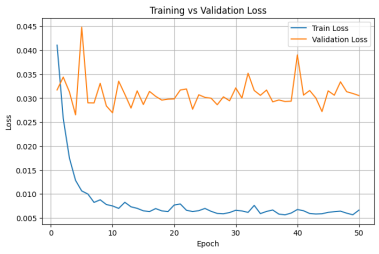

Figure 9: Training vs Validation for Oversampling

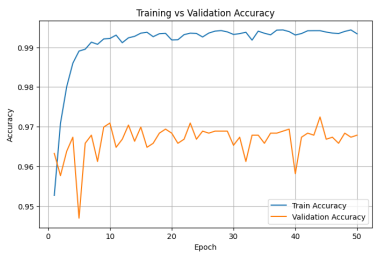

Figure 10: Training vs Validation Accuracy graph for Oversampling

To address this limitation, we adopt a *hybrid sampling* strategy inspired by Seiffert et al. [3], who demonstrate that combining oversampling and undersampling can improve classifier performance on imbalanced datasets. In our setting, hybrid sampling consists of two coordinated steps:

1. **Oversampling** images that contain at least one football, increasing the presence of minority-class (ball) grid cells while still preserving the co-occurring player cells naturally present in such images.
2. **Light undersampling** of player-only images, reducing the dominance of majority-class (player) contexts without eliminating their variability or depriving the model of purely player-based scenarios.

This approach is particularly well-suited for our data, because ball-containing images inherently contain several players. Hybrid sampling therefore increases the representation of the minority class without removing the structural diversity of the majority class. Moreover, the method avoids artificially synthesizing examples or distorting object distribution, making it compatible with YOLO-style grid classification. By selectively rebalancing the dataset at the image level, hybrid sampling yields a more informative distribution of positive grid cells and encourages the model to learn a more reliable decision boundary for the ball class as we can see in Figure 11 as well as the graphs 12 & 13.

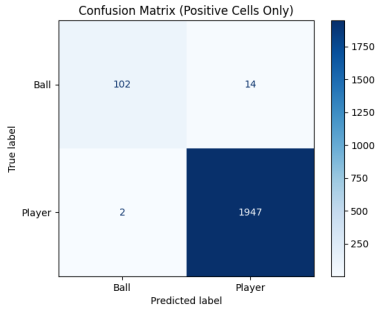

Figure 11: Confusion matrix for Overampling

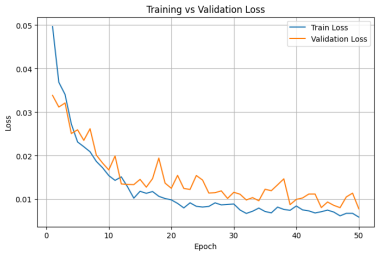

Figure 12: Training vs Validation for Oversampling

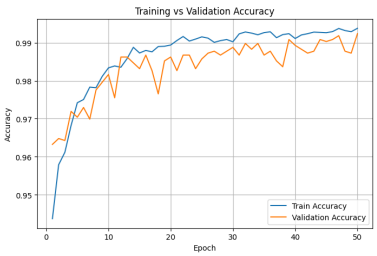

Figure 13: Training vs Validation Accuracy graph for Oversampling

3.1 Learning Rate

After applying hybrid sampling, the batches the model sees during training become more varied: some contain several ball cells, others mostly player cells. This makes optimization less stable, and a fixed learning rate can easily settle into a suboptimal local minimum dominated by the majority class. To avoid this, we perform a *learning rate range test*, which helps identify a learning rate interval where the model learns effectively.

The idea is simple: we start with a very small learning rate and increase it gradually after each batch while measuring the loss. When plotting loss against the learning rate, the region where the loss decreases most clearly indicates where the model trains best. In our case, the steep improvement appears between 10^{-5} and 10^{-4} as shown in Figure 14. This interval is later used as the lower and upper bounds for our cyclic learning schedule, allowing the optimizer to escape poor minima and converge more reliably under the rebalanced training setup.

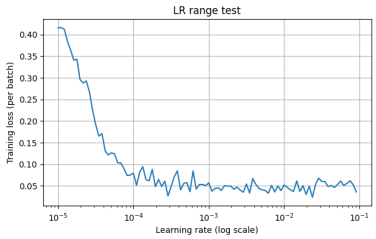

Figure 14: LR Range Test

3.2 Learning rate decay

A standard learning rate decay schedule becomes too small too early, which is problematic in our hybrid-sampled setting where minority-class signals appear unpredictably. This can cause premature convergence to a majority-class–biased solution, motivating the use of cyclic learning rates that periodically raise the learning rate and allow continued exploration. Because of these limitations, a decaying schedule is not well suited for our training setup, and this directly motivates our choice to use a cyclic learning rate, which maintains sufficient step size throughout training to handle the fluctuating batch composition introduced by hybrid sampling.

Cyclic learning

Using the learning rate range test, we identified 10^{-5} to 10^{-4} as the interval where the model learns most effectively. We therefore applied a cyclic learning rate schedule that oscillates between these two values during training. Compared to our earlier experiments with a fixed learning rate of 10^{-3}, which was often too large and caused unstable updates on the rebalanced dataset, the cyclic schedule explores the loss landscape far more efficiently. The lower bound (10^{-5}) allows fine-grained refinement, while the upper bound (10^{-4}) provides enough step size to escape poor local minima

without overshooting. This balance enables the optimizer to adapt better to the varying batch composition introduced by hybrid sampling.

As shown in Figure 16 and Figure 17, the validation curves follow the training curves much more closely than in previous experiments, indicating improved stability and better generalization. Both the loss and the accuracy are consistently lower and smoother across epochs. Most importantly, the confusion matrix in Figure 15 shows a clear improvement in recognising both classes, with significantly fewer ball misclassifications. This demonstrates that cycling between the selected learning rates not only stabilizes training but also strengthens the model's ability to find better minima than those obtained with a fixed learning rate such as 10^{-3}.

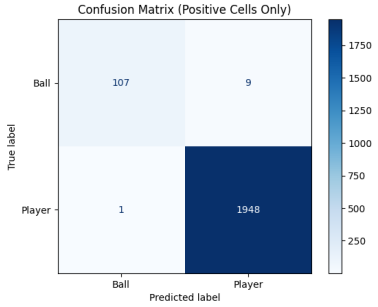

Figure 15: Confusion matrix for Hybrid sampling with cycling learning rate

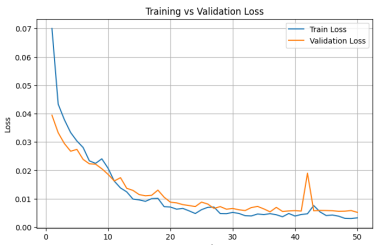

Figure 16: Training vs Validation for Hybrid sampling with cycling learning rate

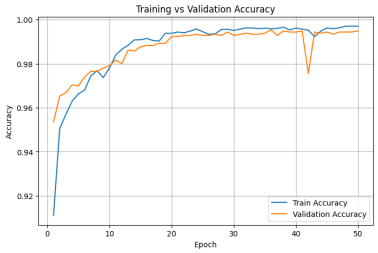

Figure 17: Training vs Validation Accuracy graph for Hybrid sampling with cycling learning rate

3.3 Hard Negative Mining

Even with rebalanced training distributions, deep learning models often converge to suboptimal decision boundaries because the majority of samples are either trivial or consistently well-classified. This problem is especially visible in grid-based classifiers, where many training examples contain only easy player-positive cells and very few regions that challenge the model's ability to recognize the minority ball class. A widely used strategy to address this issue is *hard negative mining*, where the model is explicitly trained on examples where it performs poorly. This technique forces the network to correct its systematic errors and refine its decision boundary around difficult regions of the input space [4]. We adopt a related but dataset-specific adaptation of hard negative mining. After the initial training phase, we evaluate all training images using the trained model and compute their cell-level classification accuracy. Images with perfect accuracy are discarded, while the remaining ones are ranked by descending classification error. From this ranking, we select a fixed fraction of the hardest images and construct a dedicated hard-example subset. During fine-tuning, the model is trained in two alternating passes: one over the full training set and one over the hard subset. To stabilize optimization on this noisier distribution, we reduce the learning rate before fine-tuning, transforming the second stage into a corrective refinement phase and the results can be seen in 18.

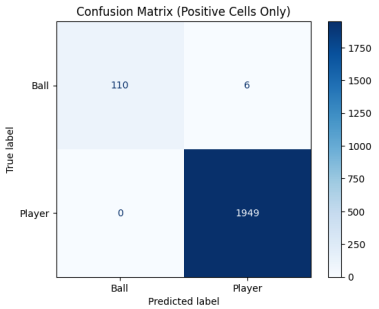

Figure 18: Confusion matrix for Hard Negative Mining

This procedure is particularly effective for our dataset, where ball-positive cells are rare and frequently overshadowed by many easy player-positive cells. Hard negative mining ensures that the model repeatedly encounters precisely those situations where it fails to distinguish ball from player, allowing it to adjust its representation toward features that discriminate the minority class. Combined hybrid sampling, hard negative mining provides a complementary mechanism that targets the model's systematic weaknesses rather than merely rebalancing the dataset.

4 Results

Table 1 summarises the performance of all methods. The baseline model achieves high player accuracy due to class imbalance but fails almost completely on the minority ball class (<56%).

Random oversampling improves ball accuracy to roughly 54% while keeping player accuracy near perfect. Hybrid sampling further increases ball accuracy to 87%, confirming that jointly oversampling ball images and lightly undersampling player-only images yields a more effective distribution shift than oversampling alone.

Adding a cyclic learning rate on top of hybrid sampling stabilises optimisation and improves ball accuracy to around 92%. Finally, a short hard-negative mining fine-tuning stage provides the best result, raising ball accuracy above 94% while maintaining >99% player accuracy.

Overall, each method contributes incrementally, and the combination of hybrid sampling, cyclic learning, and hard-negative mining gives the strongest minority-class performance in our pipeline.

Table 1: Unified comparison of all methods on the test set. Accuracy values refer to positive grid cells only.

Method	**Ball Accuracy**	**Player Accuracy**	**Overall Accuracy**
Baseline	$<$56%	$>$99%	$\sim$96–97%
Weighted Loss	$<$56%	$>$99%	$\sim$96–97%
Oversampling	$\sim$54%	$>$99%	$\sim$97%
Hybrid Sampling (fixed LR)	$\sim$87%	$>$99%	$\sim$97%
Hybrid + Cyclic LR	$\sim$92%	$>$99%	$\sim$97%
Hybrid + Cyclic LR + Hard Negative Mining	$>$94%	$>$99%	$\sim$97%

4.1 Discussion and Future Work

Across our experiments, several techniques were evaluated to improve the classification of ball and player cells. Some approaches yielded meaningful improvements, while others had limited effect. Class-weighted loss, for example, did not significantly improve minority-class performance because the batches remained dominated by player-positive cells, making the weighted gradients too infrequent to meaningfully influence training. Similarly, using a fixed learning rate of 10^{-3} produced unstable updates after hybrid sampling and did not allow the model to adapt reliably to the more varied batch composition.

In contrast, hybrid sampling reshaped the distribution of positive grid cells and produced the largest improvement in minority-class recognition. Cyclic learning between 10^{-5} and 10^{-4} further stabilized training and enabled more effective exploration of the loss landscape. Hard negative mining contributed additional refinement by focusing the model on the few remaining failure cases. Overall, the experiments showed that not all theoretically motivated techniques translate into real gains, and that sampling and optimization strategies tailored to the dataset structure had the strongest impact.

Future work can stay within the classification-only framework while exploring more complex scenarios. One direction is to extend the problem to multiple classes, which would introduce more nuanced decision boundaries and place greater demand on the sampling strategy. Another is to evaluate the model on a larger and more diverse dataset, where differences in lighting, viewpoints, and match conditions could reveal additional strengths and limitations of hybrid sampling and cyclic learning. These extensions would allow a deeper examination of how lightweight architectures handle richer classification tasks under imbalance and variability.

References

[1] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International journal of computer vision*, 88:303–338, 2010.

[2] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.

[3] Chris Seiffert, Taghi M. Khoshgoftaar, and Jason Van Hulse. Hybrid sampling for imbalanced data. In *2008 IEEE International Conference on Information Reuse and Integration*, pages 202–207, July 2008.

[4] Abhinav Shrivastava, Abhinav Gupta, and Ross Girshick. Training region-based object detectors with online hard example mining. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 761–769, 2016.

[5] Zhiqiang Zhao, Peng Zheng, Shoutao Xu, and Xindong Wu. Object detection with deep learning: A review. *IEEE Transactions on Neural Networks and Learning Systems*, 30(11):3212–3232, 2019.